

Lab-03: Branching and Merging Hands-On Lab

Aim

The aim of this lab is to understand Git branching and merging concepts. This lab demonstrates how to create a new branch, work independently without affecting the main branch, merge the changes into the main branch, and finally delete the branch after successful merging.

Objectives

- Understand Git branching and merging.
- Create a new branch in Git.
- Make changes in a separate branch.
- Merge the branch into the main branch.
- View commit history after merging.
- Delete the merged branch.

Prerequisites

- Git installed and configured.
- Existing Git repository (GitDemo).
- GitHub account.
- Command Prompt or Git Bash.

What is Branching?

A branch in Git allows developers to work on new features or changes independently without affecting the main branch. Once the work is completed and tested, the branch can be merged into the main branch.

What is Merging?

Merging combines the changes from one branch into another branch. After merging, both branches contain the same changes.

Implementation

Step 1: Open the Existing Git Repository

Navigate to the Git repository.

```
cd Desktop
```

```
cd GitDemo
```

Verify the repository status.

```
git status
```

Output:

On branch main

nothing to commit, working tree clean

Step 2: Create a New Branch

Create a new branch named **GitNewBranch**.

```
git branch GitNewBranch
```

Step 3: View Available Branches

Display all local and remote branches.

```
git branch -a
```

Output:

```
* main
GitNewBranch
remotes/origin/main
```

The * symbol indicates the currently active branch.

Step 4: Switch to the New Branch

```
git checkout GitNewBranch
```

Verify the active branch.

```
git branch
```

Output:

```
main
* GitNewBranch
```

Step 5: Create a New File

Create a new file in the branch.

```
echo This file belongs to GitNewBranch > branchfile.txt
```

Verify the file.

```
dir
```

Output:

```
branchfile.txt
welcome.txt
```

Step 6: Check Repository Status

```
git status
```

Output:

Untracked files:
branchfile.txt

Step 7: Add the File

```
git add branchfile.txt
```

Step 8: Commit the Changes

```
git commit -m "Added branchfile in GitNewBranch"
```

Example Output:

```
[GitNewBranch xxxxxx] Added branchfile in GitNewBranch  
1 file changed  
create mode 100644 branchfile.txt
```

Step 9: Verify Repository Status

```
git status
```

Output:

On branch GitNewBranch

nothing to commit, working tree clean

Step 10: Switch Back to the Main Branch

```
git checkout main
```

Step 11: Compare Both Branches

Display the differences between the main branch and the new branch.

```
git diff main GitNewBranch
```

Git displays the differences between the two branches.

Step 12: Merge the Branch

Merge the new branch into the main branch.

```
git merge GitNewBranch
```

Example Output:

```
Updating xxxxxx..xxxxxx
```

Fast-forward

```
branchfile.txt
```

Step 13: View Commit History

```
git log --oneline --graph --decorate
```

Example Output:

- * xxxxxx (HEAD -> main) Added branchfile in GitNewBranch
- * xxxxxx Added gitignore file
- * xxxxxx Added welcome.txt

Step 14: Delete the Branch

After successful merging, delete the branch.

```
git branch -d GitNewBranch
```

Example Output:

Deleted branch GitNewBranch.

Step 15: Verify Repository Status

```
git status
```

Output:

On branch main

nothing to commit, working tree clean

Step 16: Push Changes to GitHub

```
git push origin main
```

Commands Executed

```
cd Desktop
```

```
cd GitDemo
```

```
git status
```

```
git branch GitNewBranch
```

```
git branch -a
```

```
git checkout GitNewBranch
```

```
git branch
```

```
echo This file belongs to GitNewBranch > branchfile.txt
```

```
dir
```

```
git status
```

```
git add branchfile.txt
```

```
git commit -m "Added branchfile in GitNewBranch"
```

git status

git checkout main

git diff main GitNewBranch

git merge GitNewBranch

git log --oneline --graph --decorate

git branch -d GitNewBranch

git status

git push origin main

What Was Implemented

- Opened the existing Git repository.
- Created a new branch named **GitNewBranch**.
- Listed all available branches.
- Switched to the newly created branch.
- Created a new file named **branchfile.txt**.
- Added content to the file.
- Added the file to the staging area.
- Committed the changes in the new branch.
- Verified the repository status.
- Switched back to the **main** branch.
- Compared the differences between the **main** branch and **GitNewBranch**.
- Merged **GitNewBranch** into the **main** branch.
- Viewed the commit history using the graph view.
- Deleted the merged branch.
- Verified that the working tree was clean.
- Pushed the merged changes to the remote GitHub repository.

Result

The Branching and Merging Hands-On Lab was successfully completed. A new branch was created, independent changes were made and committed in that branch, and the branch was successfully merged into the main branch. After verifying the commit history, the merged branch was deleted,

and the updated repository was pushed to GitHub, demonstrating the complete Git branching and merging workflow.

```
Command Prompt
C:\Users\vanka\Desktop\GitDemo>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

C:\Users\vanka\Desktop\GitDemo>git diff main GitNewBranch
diff --git a/branchfile.txt b/branchfile.txt
new file mode 100644
index 0000000..1b391a7
--- /dev/null
+++ b/branchfile.txt
@@ -0,0 +1 @@
+This file is created inside GitNewBranch

C:\Users\vanka\Desktop\GitDemo>git merge GitNewBranch
Updating 59102ff..465d088
Fast-forward
 branchfile.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 branchfile.txt

C:\Users\vanka\Desktop\GitDemo>git log --oneline --graph --decorate
* 465d088 (HEAD -> main, GitNewBranch) Added branch file in GitNewBranch
* 59102ff (origin/main) Added gitignore file
* f5cd28d Added welcome.txt

C:\Users\vanka\Desktop\GitDemo>git branch -d GitNewBranch
Deleted branch GitNewBranch (was 465d088).

C:\Users\vanka\Desktop\GitDemo>git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

C:\Users\vanka\Desktop\GitDemo>git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 394 bytes | 197.00 KiB/s, done.
```